

TÀI LIỆU HƯỚNG DẪN HỆ THỐNG HỎI ĐÁP PHÁP LUẬT VIỆT NAM R2AI Legal QA System

Dành cho doanh nghiệp vừa và nhỏ (SME)

Ngày 30 tháng 6 năm 2026

Mục lục

1	Tổng quan hệ thống	2
1.1	Hai giai đoạn xử lý	2
1.2	Kết quả xử lý	2
2	Kiến trúc pipeline	2
2.1	Retrieval Pipeline (pipeline-corpus.ipynb)	2
2.2	LLM Generation Pipeline (llm-gen.ipynb)	3
3	Mô hình sử dụng	3
3.1	BAAI/bge-m3 — Dense Embedder	3
3.2	BAAI/bge-reranker-v2-m3 — Cross-Encoder Reranker	4
3.3	Qwen/Qwen3-14B — LLM Answer Generation	4
3.4	Tổng hợp mô hình	5
4	Dữ liệu	5
4.1	Dữ liệu cho Retrieval Pipeline (Kaggle Dataset)	5
4.2	Dữ liệu cho LLM Pipeline (Google Drive)	5
5	Hướng dẫn tái hiện (Reproduce)	7
5.1	Yêu cầu môi trường phần cứng và phần mềm	7
5.2	Cài đặt thư viện phụ thuộc	7
5.3	Thiết lập Kaggle Notebook — Bước 1	7
5.4	Thiết lập Kaggle Notebook — Bước 2	8
6	Danh sách thư viện và phụ thuộc	8
7	Format output và kiểm tra dữ liệu đầu ra	10

1 Tổng quan hệ thống

R2AI là hệ thống hỏi đáp pháp luật Việt Nam chuyên biệt cho doanh nghiệp vừa và nhỏ (SME). Hệ thống tự động trả lời các câu hỏi pháp lý bằng cách tìm kiếm điều luật liên quan từ kho pháp luật và sinh câu trả lời có trích dẫn chính xác.

1.1 Hai giai đoạn xử lý

Bảng 1: Các giai đoạn xử lý chính của hệ thống R2AI

Giai đoạn	Notebook	Mô tả	Output
Bước 1	pipeline-corpus.ipynb	Retrieval: tìm điều luật liên quan cho 2,000 câu hỏi bằng BM25 + Dense + Rerank	retrieval_v6.json
Bước 2	llm-gen.ipynb	LLM Generation: Qwen3-14B sinh câu trả lời tiếng Việt + trích dẫn điều luật	results.json + .zip

1.2 Kết quả xử lý

Bảng 2: Các chỉ số thống kê kết quả xử lý

Chỉ số	Giá trị
Số câu hỏi xử lý	2,000 câu
Kho điều luật	6,151 điều (62 văn bản pháp luật)
Clause chunks dense index	41,746 chunks
Số điều luật trả về tối đa / câu	7 điều (MAX_K=7)
Số điều luật trả về tối thiểu / câu	1 điều (MIN_K=1)

2 Kiến trúc pipeline

2.1 Retrieval Pipeline (pipeline-corpus.ipynb)

Pipeline retrieval gồm 4 tầng kết hợp tuần tự và song song nâng cao:

- BM25 Sparse Retrieval:** Tiến hành tokenize bằng `underthesea` (word-level) kết hợp với 4-char n-gram, các tham số cấu hình bao gồm $k_1 = 1.5$, $b = 0.4$ (đã được tinh chỉnh chuyên biệt cho văn bản pháp luật), trả về kết quả top-100 điều luật phù hợp nhất.
- Dense Clause-Level Retrieval:** Thực hiện encode từng clause (≤ 256 tokens) bằng mô hình BGE-M3, tính toán độ tương đồng thông qua cosine similarity, lấy top-200 clauses $\rightarrow$ áp dụng chiến lược MaxRank $\rightarrow$ trích xuất ra top-50 articles. Việc thực hiện encode ở cấp độ clause thay vì toàn bộ full article giúp không gian vector tập trung ngữ nghĩa chính xác hơn.
- Query Enrichment:** Sử dụng file cấu hình `r2ai_enrichment.jsonl` nhằm mục đích mở rộng query gốc thành các biến thể đa dạng với keywords và sub-queries bổ trợ. Mỗi variant này sẽ đóng góp thêm một BM25 leg và một Dense leg trực tiếp vào hệ thống xếp hạng chung RRF.
- RRF Fusion ($k = 60$):** Thực hiện fusion kết hợp tất cả các nhánh ranking legs bao gồm (BM25 main + Dense main + các enrichment legs bổ sung) tạo thành một candidate pool cô đọng chứa 50 điều luật tiềm năng.

5. **BGE-reranker-v2-m3 (Cross-Encoder):** Áp dụng kỹ thuật Sliding Window MaxP — thực hiện chia nhỏ văn bản đầy đủ (`full_text`) thành các chunk có độ dài 1,200 ký tự (độ chồng lấp overlap là 200 ký tự), chấm điểm (score) độc lập cho từng cặp (*query*, *chunk*), sau đó lấy giá trị lớn nhất MaxP đại diện cho mỗi điều luật. Cuối cùng, áp dụng bộ lọc **Adaptive Soft-Floor Cutoff** nhằm giữ lại những điều luật có điểm số $\geq \text{ABS_FLOOR} = 0.38$ hoặc nằm trong ngưỡng tối ưu `dynamic_gap` của top-1.

2.2 LLM Generation Pipeline (`llm-gen.ipynb`)

6. **Voting Ensemble:** Thực hiện tải và xử lý đồng thời 10 file zip chứa kết quả submission cũ, đếm tổng số lượng vote cho từng điều luật cụ thể tương ứng theo từng câu hỏi → sàng lọc và chọn ra top-10 điều luật ứng viên sáng giá có số lượng vote ≥ 1 .
7. **Corpus Lookup:** Tiến hành tra cứu nội dung văn bản đầy đủ (`full_text`) của các điều luật từ kho dữ liệu chính SME corpus (primary) kết hợp tra cứu chéo trên `thinhng0` fallback corpus trong trường hợp không tìm thấy dữ liệu gốc.
8. **Qwen3-14B Inference:** Xây dựng cấu trúc prompt hoàn chỉnh tích hợp cùng system prompt pháp lý nghiêm ngặt + danh sách đầy đủ các điều luật đã tra cứu + câu hỏi đầu vào. Mô hình xử lý dữ liệu và trả về kết quả chuẩn hóa theo cấu trúc định sẵn gồm `CITED` + `ANSWER` + `END`.
9. **Checkpoint & Export:** Ghi nhận dữ liệu liên tục vào JSONL checkpoint ngay sau khi xử lý xong từng câu hỏi (đảm bảo tính năng an toàn để có thể resume/khôi phục tiến trình khi xảy ra sự cố). Sau cùng xuất ra file kết quả tổng hợp dạng `results.json` và đóng gói file nén `.zip`.

3 Mô hình sử dụng

Hệ thống sử dụng tích hợp 3 mô hình học sâu pre-trained hiện đại từ nền tảng HuggingFace, tất cả đều thuộc dạng mã nguồn mở và được cộng đồng tối ưu hóa cao.

3.1 BAAI/bge-m3 — Dense Embedder

Bảng 3: Thuộc tính kỹ thuật của mô hình BAAI/bge-m3

Thuộc tính	Thông tin chi tiết
Tên model	BAAI/bge-m3
Phiên bản checkpoint	Revision: main (latest stable)
Kích thước	~2.27 GB (safetensors)
Kiến trúc	XLM-RoBERTa-Large (560M parameters)
Độ dài đầu vào	Tối đa 8,192 tokens; cấu hình thực tế dùng 256 tokens/clause trong hệ thống này
Ngôn ngữ	Đa ngôn ngữ (hỗ trợ hơn 100+ ngôn ngữ, tối ưu tốt cho tiếng Việt)
Precision sử dụng	float16 (<code>model.half()</code>) giúp tiết kiệm tối đa tài nguyên VRAM
Mục đích	Encode các clause chunks thành dense vector; encode query và tính toán cosine similarity

Đường link checkpoint chính thức: <https://huggingface.co/BAAI/bge-m3>

Cách tải và sử dụng checkpoint trong mã nguồn:

```
# Tự động tải khi chạy notebook (yêu cầu kết nối Internet):
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('BAAI/bge-m3', device='cuda:0')
embedder.max_seq_length = 256
embedder.half() # Sử dụng fp16 để giảm thiểu dung lượng VRAM
```

```
# Hoặc tiến hành tải thủ công sẵn về máy local:
# huggingface-cli download BAAI/bge-m3 --local-dir ./models/bge-m3
```

3.2 BAAI/bge-reranker-v2-m3 — Cross-Encoder Reranker

Bảng 4: Thuộc tính kỹ thuật của mô hình BAAI/bge-reranker-v2-m3

Thuộc tính	Thông tin chi tiết
Tên model	BAAI/bge-reranker-v2-m3
Phiên bản checkpoint	Revision: main (latest stable)
Kích thước	~2.27 GB (safetensors)
Kiến trúc	XLNet-RoBERTa-Large fine-tuned chuyên biệt cho tác vụ reranking (cross-encoder)
Độ dài đầu vào	Tối đa 512 tokens (bao gồm tổng thể query + passage)
Ngôn ngữ	Đa ngôn ngữ, hỗ trợ tối ưu cao cho tiếng Việt
Precision sử dụng	float16 (model.half())
Mục đích	Rerank lại candidate pool 50 điều luật với thuật toán Sliding Window MaxP; thực hiện chấm điểm score chuẩn xác cho mỗi cặp (query, chunk)

Đường link checkpoint chính thức: <https://huggingface.co/BAAI/bge-reranker-v2-m3>

Cách tải và sử dụng checkpoint trong mã nguồn:

```
# Tự động tải khi chạy notebook (yêu cầu kết nối Internet):
from sentence_transformers import CrossEncoder

reranker = CrossEncoder('BAAI/bge-reranker-v2-m3', max_length=512, device='cuda:1')
reranker.model.half() # Cấu hình định dạng fp16

# Tiến hành tính toán Score cho các cặp (query, passage):
scores = reranker.predict([[query, passage1], [query, passage2]])

# Hoặc tiến hành tải thủ công sẵn về máy:
# huggingface-cli download BAAI/bge-reranker-v2-m3 --local-dir ./models/bge-reranker-v2-m3
```

3.3 Qwen/Qwen3-14B — LLM Answer Generation

Đường link checkpoint chính thức: <https://huggingface.co/Qwen/Qwen3-14B>

Cách tải và sử dụng checkpoint qua chế độ 4-bit quantization:

```
# Tự động tải khi khởi chạy notebook (Dung lượng tải file nén ~9 GB):
from transformers import AutoModelForCausalLM, AutoTokenizer,
    BitsAndBytesConfig
import torch

model_id = 'Qwen/Qwen3-14B'

# Thiết lập cấu hình 4-bit NF4 quantization (giúp tiết kiệm VRAM, chạy tốt trên kiến trúc GPU T4 16GB)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
```

Bảng 5: Thuộc tính kỹ thuật của siêu mô hình Qwen/Qwen3-14B

Thuộc tính	Thông tin chi tiết
Tên model	Qwen/Qwen3-14B
Phiên bản checkpoint	Revision: main (latest stable, released 2025)
Kích thước	~28 GB (định dạng gốc fp16/BF16) / ~9 GB (khi qua cấu hình 4-bit NF4 quantization)
Kiến trúc	Transformer decoder, cấu hình 14B parameters, hỗ trợ ngữ cảnh 32K context cực lớn
Context window	32,768 tokens; cấu trúc hệ thống này giới hạn MAX_INPUT_TOKENS=24,576
Ngôn ngữ	Hỗ trợ đa ngôn ngữ, sinh văn bản tiếng Việt chất lượng cao và tự nhiên
Quantization	Tối ưu hóa qua 4-bit NF4 (bitsandbytes) + double quant, cài đặt <code>compute_dtype=torch.float16</code>
Thinking mode	Mô hình hỗ trợ hai chế độ <code>/think</code> và <code>/no_think</code> ; cấu trúc hệ thống thiết lập dùng chế độ <code>/no_think</code> (THINKING='off') để tối ưu tốc độ sinh từ tối đa
Mục đích	Chịu trách nhiệm sinh câu trả lời pháp lý tiếng Việt kèm trích dẫn điều luật logic từ candidate pool

```

    bnb_4bit_quant_type='nf4',
)

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map='auto',
)
model.eval()

# Hoặc nếu muốn thực hiện download thủ công:
# huggingface-cli download Qwen/Qwen3-14B --local-dir ./models/qwen3-14b

```

3.4 Tổng hợp mô hình

Bảng 6: Bảng tổng hợp so sánh các mô hình trong kiến trúc R2AI

Mô hình	Phiên bản	Kích thước	VRAM cần	Đường link HuggingFace
BAAI/bge-m3	main (latest)	2.27 GB	~3 GB	huggingface.co/BAAI/bge-m3
BAAI/bge-reranker-v2-m3	main (latest)	2.27 GB	~3 GB	huggingface.co/BAAI/bge-reranker-v2-m3
Qwen/Qwen3-14B	main (latest)	~28 GB (fp16)	~9 GB (4-bit)	huggingface.co/Qwen/Qwen3-14B

4 Dữ liệu

4.1 Dữ liệu cho Retrieval Pipeline (Kaggle Dataset)

Địa chỉ Dataset trên nền tảng Kaggle: [letuano5/r2ai-corpus](#)

4.2 Dữ liệu cho LLM Pipeline (Google Drive)

Nguồn tải dữ liệu gốc từ: **Google Drive — R2AI Data**

Bảng 7: Chi tiết các tệp dữ liệu dùng cho Retrieval Pipeline

Tên File	Dung lượng	Mô tả chi tiết nội dung
corpus_luat_sme_merged_v3.jsonl	14.18 MB	Kho dữ liệu luật SME chính — bao gồm ~5,658 điều luật thuộc 62 văn bản pháp luật hiện hành tại Việt Nam
patch_laws_articles.jsonl	820 kB	Chứa thông tin 5 bộ luật bổ sung giai đoạn 2024-2025 (cập nhật thêm 273 điều luật mới)
sme_clauses_v4.jsonl	51.57 MB	Tổng cộng 41,746 bộ dữ liệu clause chunks phân rã từ bộ SME corpus (phục vụ trực tiếp cho việc đánh chỉ mục dense index)
patch_laws_clauses.jsonl	2.89 MB	Tổng hợp thông tin các Clause chunks thuộc nhóm các luật sửa đổi bổ sung (patch laws)
R2AIStage1DATA.json	531 kB	Tập dữ liệu test set bao gồm đúng 2,000 câu hỏi tình huống thực tế về pháp luật
r2ai_enrichment.jsonl (tùy chọn)	2.39 MB	Bộ nhớ cache cho Query enrichment: lưu trữ sẵn keywords + sub-queries tối ưu hóa cho mỗi câu hỏi

Bảng 8: Cấu trúc ánh xạ và mô tả các tệp dữ liệu dùng cho LLM Pipeline

File / Thư mục gốc	Kaggle Dataset đích	Mô tả chi tiết mục đích
corpus_luat_sme_merged_v3.jsonl	r2ai-corpus (dùng chung)	Bộ SME corpus — đóng vai trò làm primary lookup chính xác cho cấu trúc của LLM
th1nhng0_articles_v2.jsonl	r2ai-corpus	Fallback article corpus dung lượng lớn (~517 MB) — hỗ trợ tra cứu mở rộng khi hệ thống primary không tìm thấy từ khóa
old_submission/*.zip (10 files)	r2ai-old-submissions (tạo mới)	Tập hợp 10 file dữ liệu submission cũ — phục vụ thuật toán voting ensemble nhằm chất lọc chính xác điều luật ứng viên

5 Hướng dẫn tái hiện (Reproduce)

5.1 Yêu cầu môi trường phần cứng và phần mềm

Bảng 9: Bảng đặc tả kỹ thuật và yêu cầu môi trường hệ thống

Thành phần	Chi tiết yêu cầu kỹ thuật tối thiểu
Nền tảng thực thi	Hệ thống Kaggle Notebooks (khuyến nghị ưu tiên) hoặc hệ thống máy chủ vật lý được trang bị cấu hình GPU hỗ trợ CUDA mạnh mẽ
Runtime Bước 1	Cấu hình phần cứng chạy GPU T4 x2 hoặc GPU P100 (đảm bảo tổng dung lượng đạt ≥ 16 GB VRAM)
Runtime Bước 2	Khuyến nghị GPU T4 x2, P100 hoặc card chuyên dụng A100 (≥ 16 GB VRAM khi thực hiện load mô hình ở chế độ 4-bit quant)
Môi trường Python	Phiên bản ổn định 3.10+ (Hệ thống Kaggle mặc định hiện tại đã cung cấp đủ)
Kết nối mạng Internet	Trạng thái BẬT (ON) — bắt buộc phải có để hệ thống tải tự động model từ HuggingFace
Dung lượng cache	Đạt tối thiểu ≥ 20 GB (Dành cho việc lưu trữ các models: ~ 7 GB, lưu trữ tập dữ liệu clause embeddings: ~ 160 MB)

5.2 Cài đặt thư viện phụ thuộc

Bước 1 — Cài đặt môi trường cho Retrieval (Thực thi mã lệnh này tại Cell 1 của notebook `pipeline-corpus.ipynb`):

```
pip install -q sentence-transformers rank_bm25 underthesea
```

Bước 2 — Cài đặt môi trường cho LLM (Cần thiết lập biến cấu hình `INSTALL_PACKAGES = True` trong cell đầu tiên của tệp `llm-gen.ipynb`):

```
pip install -q --upgrade transformers>=4.51 accelerate>=0.33 bitsandbytes  
>=0.43 tqdm
```

5.3 Thiết lập Kaggle Notebook — Bước 1

- Thực hiện khởi tạo một Kaggle Notebook hoàn toàn mới, sau đó upload tệp cấu hình mã nguồn `pipeline-corpus.ipynb` lên hệ thống.
- Truy cập vào mục **Settings** > mục **Accelerator**: Tiến hành lựa chọn cấu hình **GPU T4 x2** (hoặc có thể thay thế bằng dòng card **P100**).
- Kiểm tra kỹ mục **Settings** > cài đặt thành phần **Internet**: Chuyển sang trạng thái **BẬT (ON)**.
- Nhấn chọn nút **Add Data**: Tiến hành nhập tìm kiếm chính xác bộ dataset mang tên `letuano5/r2ai-corpus`, sau đó bấm nút **Add**.
- Thực hiện chạy thử nghiệm và kiểm tra xác nhận tại vị trí **Cell 3** đảm bảo hệ thống hiển thị chính xác đầy đủ 5 dấu tích xanh () đại diện thành công cho 5 file data đầu vào.
- Kích hoạt tính năng **Run All** — để hệ thống bắt đầu biên dịch và chạy tuần tự liên mạch từ vị trí **Cell 1** cho tới **Cell 11**.

5.4 Thiết lập Kaggle Notebook — Bước 2

1. Tải toàn bộ các file dữ liệu cần thiết từ link chia sẻ Google Drive (đã mô tả tại mục 4.2), sau đó thực hiện upload chúng lên thành 2 phân vùng Kaggle Dataset độc lập như sau:
 - **Dataset r2ai-corpus:** Tiến hành thêm trực tiếp tệp văn bản `th1nhng0_articles_v2.jsonl` vào trong phân vùng dataset `letuano5/r2ai-corpus`.
 - **Dataset r2ai-old-submissions** (thực hiện tạo mới hoàn toàn): Upload đồng thời cả 10 tệp định dạng nén `.zip` của submission cũ lên hệ thống.
2. Tiến hành khởi tạo một thực thể Kaggle Notebook mới khác, upload tệp mã nguồn ứng dụng `llm-gen.ipynb`.
3. Truy cập vào mục **Settings** > mục **Accelerator**: Lựa chọn dòng card xử lý đồ họa nâng cao **GPU T4 x2** hoặc card hiệu năng cao **A100**.
4. Kiểm tra xác thực mục **Settings** > phân hệ **Internet**: Xác nhận đã ở trạng thái **BẬT (ON)**.
5. Click chọn **Add Data**: Thêm đồng bộ cùng một lúc cả 2 bộ dữ liệu cấu thành (`r2ai-corpus` và `r2ai-old-submissions`).
6. Kiểm tra rà soát kỹ lưỡng các khai báo đường dẫn thư mục trong cell config hệ thống — cam kết phải khớp chuẩn xác với tên bộ định danh dataset Kaggle của bạn:

```
SUBMISSIONS_DIR = '/kaggle/input/r2ai-old-submissions'
SME_MERGED      = '/kaggle/input/r2ai-corpus/corpus_luat_sme_merged_v3.jsonl'
TH_ARTICLES     = '/kaggle/input/r2ai-corpus/th1nhng0_articles_v2.jsonl'
```

7. Nhấn chọn lệnh **Run All** — để hệ thống bắt đầu khởi chạy xử lý mã nguồn tuần tự từ đầu cho tới cuối trang. Khoảng thời gian đo đạc ước tính: Mất từ **3 đến 6 tiếng đồng hồ** để hoàn tất xử lý trọn vẹn cho cả thấy 2,000 câu hỏi dữ liệu đầu vào.

6 Danh sách thư viện và phụ thuộc

Bảng 10: Bảng kê chi tiết các thư viện phụ thuộc và phiên bản yêu cầu

Thư viện	Phiên bản tối thiểu	Mục đích sử dụng chính	Notebook áp dụng
torch	$\geq 2.1.0$	Thực vụ tính toán GPU inference cho tất cả mô hình	Cả hai tệp
transformers	$\geq 4.51.0$	Hỗ trợ cơ chế Load và thực hiện tác vụ inference Qwen3-14B	llm-gen
accelerate	$\geq 0.33.0$	Hỗ trợ kiến trúc Multi-GPU thông qua <code>device_map='auto'</code>	llm-gen
bitsandbytes	$\geq 0.43.0$	Thực hiện cơ chế nén 4-bit NF4 quantization học sâu	llm-gen
sentence-transformers	$\geq 2.7.0$	Tích hợp bộ BGE-M3 embedder + module chấm điểm BGE-reranker-v2-m3	pipeline-corpus
rank-bm25	$\geq 0.2.2$	Khởi chạy thuật toán tìm kiếm thừa thớt BM25Okapi sparse retrieval	pipeline-corpus
underthesea	$\geq 6.8.0$	Thư viện tách từ tiếng Việt chuẩn hóa (Vietnamese word tokenizer)	pipeline-corpus
numpy	$\geq 1.24.0$	Xử lý các phép toán trên vector quy mô lớn (cosine sim, argsort)	pipeline-corpus
tqdm	$\geq 4.66.0$	Hiển thị thanh tiến trình trực quan (Progress bars)	Cả hai tệp
json, zipfile, pathlib, hashlib, pickle	Thư viện chuẩn (stdlib)	Chịu trách nhiệm các tác vụ quản lý File I/O, bộ nhớ đệm caching, đóng gói ZIP	Cả hai tệp

7 Format output và kiểm tra dữ liệu đầu ra

Mỗi một phần tử dữ liệu bản ghi được cấu thành bên trong tệp kết quả đầu ra `results.json` bắt buộc phải tuân thủ nghiêm ngặt theo đúng cấu trúc biểu diễn JSON format tiêu chuẩn dưới đây:

```
{
  "id": 1,
  "question": "Câu hỏi pháp luật tiếng Việt...",
  "answer": "Căn cứ Điều X văn bản Y..., doanh nghiệp cần...",
  "relevant_docs": ["doc_id|Tên văn bản"],
  "relevant_articles": ["doc_id|Tên văn bản|Điều X"]
}
```

Các điều kiện ràng buộc format (Sẽ được hệ thống tự động kiểm tra định kỳ trong cell validate của notebook `llm-gen.ipynb`):

- **relevant_articles:** Yêu cầu bắt buộc mỗi phần tử chứa bên trong danh sách phải định dạng hiển thị chứa đúng chính xác **2 ký tự dấu gạch đứng (|)** (Bao gồm phân rõ đủ 3 phần thông tin cấu thành: `doc_id` | tên của văn bản pháp luật | điều luật áp dụng).
- **relevant_docs:** Yêu cầu mỗi phần tử chứa bên trong danh sách phải chứa đúng chính xác duy nhất **1 ký tự dấu gạch đứng (|)** (Bao gồm phân tách rõ ràng làm 2 phần thông tin: `doc_id` | tên của văn bản pháp luật đầy đủ).
- Trường dữ liệu `id` cam kết bắt buộc phải là định dạng số nguyên (*integer*), trường `question` phải là định dạng chuỗi văn bản ký tự (*string*) và tuyệt đối không được phép để trống, trường `answer` phải định dạng chuỗi văn bản ký tự (*string*).